Project - High Level Design

on

Non-Profit Triage Support Agent

Course Name: Datagami Skill Based Course

Institution Name: Medicaps University

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Devansh Gupta	EN22CS301325
2	Avantika Gupta	EN22CS301242
3	Chiran Vikram Shahi	EN22CS301301
4	Bharat Rathore	EN22CS301260
5	Deepali Pastariya	EN22CS301318

Group Name: 11D3

Project Number: AAI-35 Industry

Mentor Name: Mr. Ashwin

krishnan

University Mentor Name: Maya Yadav

Academic Year: 2025-26

Table of Contents

1. Introduction.	1.1. Scope of the document. 1.2. Intended Audience 1.3. System overview.
2. System Design.	2.1. Application Design 2.2. Process Flow. 2.3. Information Flow. 2.4. Components Design 2.5. Key Design Considerations 2.6. API Catalogue.
3. Data Design.	3.1. Data Model 3.2. Data Access Mechanism 3.3. Data Retention Policies 3.4. Data Migration
4. Interfaces	
5. State and Session Management	
6. Caching	
7. Non-Functional Requirements	7.1. Security Aspects 7.2. Performance Aspects
8. References	

1. INTRODUCTION

The Non-Profit Triage Support Agent is an intelligent, AI-driven system designed to automate and streamline the processing of incoming communications for non-profit organizations. These communications may include donation inquiries, complaints, return requests, emergency assistance requests, and general queries.

Non-profit organizations often receive a high volume of diverse messages, making manual handling inefficient and prone to delays. This system addresses the challenge by leveraging **Large Language Models (LLMs)** to automatically analyze and respond to user messages.

The system performs multiple tasks in a unified pipeline:

- Intent classification
- Urgency detection
- Entity extraction
- Professional response generation

A **rule-based Decision Engine** is incorporated to refine LLM outputs and ensure reliability.

1.1 Scope of the Document

This document presents the **High-Level Design (HLD)** of the Non-Profit Triage Support Agent. The primary goal of this document is to provide a comprehensive overview of the system's architecture, components, and overall workflow without delving into low-level implementation specifics.

The scope of this document includes:

- A detailed description of the **system architecture**, including all major components and their interactions
- An explanation of the **core modules**, such as classification, entity extraction, and response generation
- The **data flow** across the system, illustrating how input messages are processed and transformed into meaningful outputs
- An overview of the **technologies, frameworks, and models** used in the system
- A discussion of the system's **functional capabilities** and design considerations

However, this document does **not** cover:

- Detailed code-level implementation
- Database schema design (as the current system is stateless)
- Deployment scripts or infrastructure configuration in depth

1.2 Intended Audience

This document is intended for:

- **Developers** – to understand system structure and components
- **AI/ML engineers** – to review model usage and design
- **Project evaluators** – for academic or technical assessment
- **Non-profit stakeholders** – to understand system capabilities

1.3 System Overview

The Non-Profit Triage Support Agent is a **web-based application** that processes user messages through an AI pipeline.

Main Functions:

- Accepts incoming messages from users
- Classifies the **intent** (e.g., donation, complaint, request)
- Determines the **urgency level** (low, medium, high)
- Extracts **key entities** (such as money, dates, and locations)
- Generates a **polite AI-based response**

Workflow:

1. User submits a message
2. System analyzes it using NLP models
3. Results are generated (intent, urgency, entities)
4. A response is automatically created and displayed

The system uses:

- **LLaMA Models (via Groq API)**
- **Python**
- **Streamlit**

2. SYSTEM DESIGN

The **System Design** section describes the overall structure and organization of the Non-Profit Triage Support Agent. It explains how different components of the system are designed, how they interact with each other, and how the application processes user input to generate meaningful outputs.

2.1 Application Design

The system follows a **modular architecture** where each functionality is separated into independent components. The application is built using **Streamlit** as the frontend and integrates multiple AI models for backend processing.

The main modules include:

- User Interface (UI)
- Triage Agent (core logic)
- NLP Processing using LLM (classification & entity extraction)
- Response Generation (Generative AI)

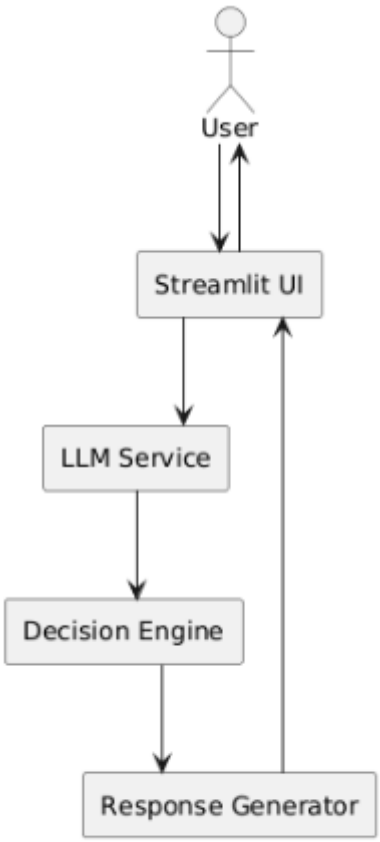

Figure 2.1.1 Application Design

2.2 Process Flow

1. User enters a message in the UI
2. Message is sent to the Triage Agent
3. Intent classification is performed

4. Urgency level is determined
5. Entities are extracted from the text
6. AI-generated response is created
7. Results are displayed to the user

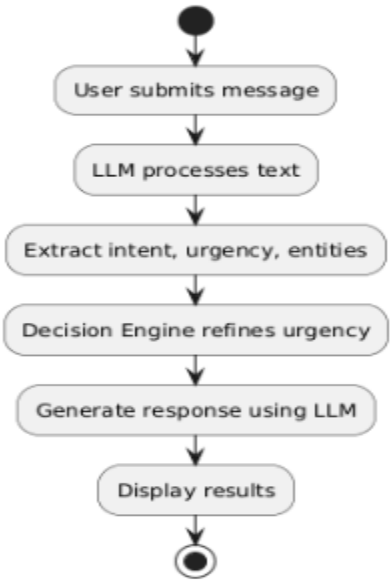

Figure 2.2.1 Process Flow Design

2.3 Information Flow

- Input: User message (unstructured text)
- Processing:
  - Classification → structured labels
  - Entity extraction → structured data
- Output:
  - Urgency
  - Entities
  - Generated response

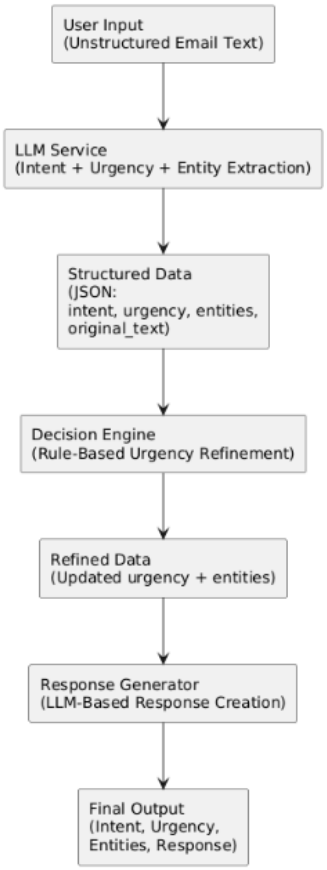

Figure 2.3.1 Information Flow Design

2.4 Components Design

1. User Interface (Streamlit)

- Provides input field
- Displays results
- Interactive UI

2. LLM Service

- Connects to Groq API
- Performs:
  - Intent classification
  - Urgency detection
  - Entity extraction
- Returns structured JSON

3. Decision Engine

- Applies rule-based refinement

- Ensures:
  - emergency → high urgency
  - complaint → minimum medium
- Improves reliability

4. Response Generator

- Uses LLM
- Takes:
  - original text
  - structured data
- Generates professional response

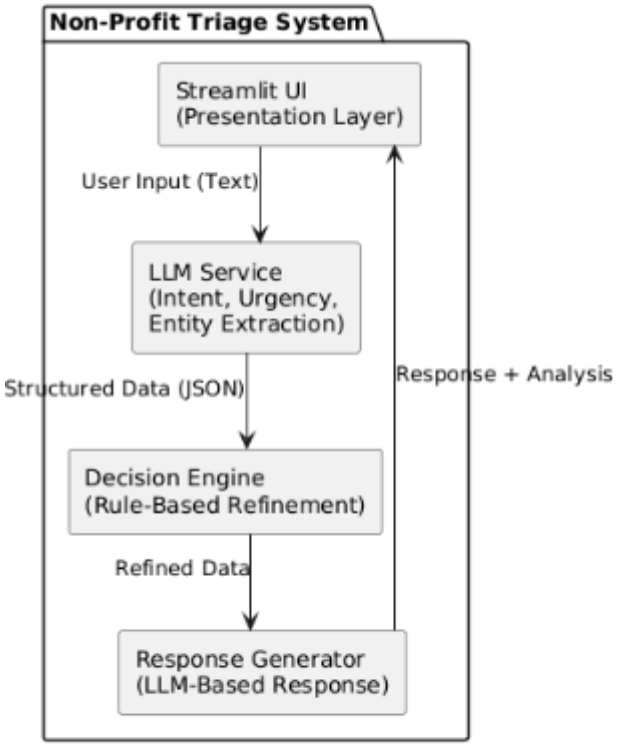

Figure 2.4.1 Components Design

2.5 Key Design Considerations

1. **Modularity** – Easy to update components
2. **Scalability** – Can handle more users/messages
3. **Usability** – Simple UI for non-technical users
4. **Accuracy** – Reliable AI models
5. **Performance** – Fast response time

2.6 API Catalogue

API	Purpose
Groq API	LLM processing
Streamlit	UI rendering

3. Data Design

The **Data Design** section outlines how data is handled within the system. It describes the structure of input and output data, how data is processed and accessed, and the policies related to data storage and retention.

3.1 Data Model

Input Data:

- **Message (Text):**
  A user-provided message containing a query, request, or information related to non-profit activities.

Output Data:

After processing, the system generates the following structured outputs:

- **Intent (String):**
  Represents the purpose of the message (e.g., donation inquiry, complaint, support request).
- **Urgency (String):**
  Indicates the priority level of the message (low, medium, or high).
- **Entities (List):**
  A collection of extracted key elements such as names, dates, monetary values, and locations.
- **Response (Text):**
  A context-aware, AI-generated reply tailored to the input message.

3.2 Data Access Mechanism

The system follows a **real-time data processing approach**:

- No traditional database is used for storing data
- All processing occurs in-memory during runtime
- Data is accessed directly within the application pipeline

This approach ensures:

- Faster processing speed
- Reduced system complexity
- Minimal infrastructure requirements

3.3 Data Retention Policies

- No user messages are stored permanently
- Data is retained only for the duration of the active session

3.4 Data Migration

Currently, **data migration is not applicable** because the system does not use persistent storage.

However, in future enhancements:

- A database may be introduced for storing message history
- Migration strategies may include transferring in-memory data to structured databases
- Data backup and synchronization mechanisms may be implemented

4. Interfaces

The **Interfaces** section defines how users interact with the system and how the system presents results.

- **Text Input Field:**
  Allows users to enter or paste incoming support messages
- **Analyze Button:**
  Triggers the processing pipeline
- **Output Display Sections:**
  Displays:
  - Detected intent
  - Urgency level
  - Extracted entities
  - AI-generated response

Interface Characteristics:

- Simple and user-friendly design
- No technical expertise required
- Interactive and responsive layout

5. State and Session Management

Key Features:

- Stores temporary inputs such as sample messages
- Maintains state during a single user session
- Ensures smooth interaction without losing data on refresh

Limitations:

- No long-term session persistence
- Data is lost when the session ends or the page is refreshed

6. Caching

- Optional caching for:
- LLM client
- Improves performance

Benefits of Caching:

- Faster execution time
- Avoids repeated loading of heavy models
- Reduces computational overhead
- Improves overall user experience

7. Non-Functional Requirements

The **Non-Functional Requirements** define the quality attributes of the system, ensuring it performs efficiently, securely, and reliably.

7.1 Security Aspects

The system incorporates basic security measures to protect data and external integrations:

- **Secure API Key Handling:**
  API keys (e.g., for generative AI) should be stored securely and not exposed in code
- **No Sensitive Data Storage:**
  User messages are not stored permanently, reducing privacy risks
- **Secure Communication:**
  All API interactions should use secure protocols such as HTTPS

7.2 Performance Aspects

The system is optimized for **fast and efficient performance**:

- **Low Response Time:**
  Results are generated within a few seconds
- **Efficient Model Usage:**
  Cached models reduce loading time
- **Real-Time Processing:**
  Immediate analysis without delays
- **Scalability Consideration:**
  The system can be extended to handle larger workloads with optimization

8. References

- Groq API Documentation
- LLaMA Model Docs
- Streamlit Documentation
- UML Standards